

1. The following code snippets are legal.
 - (a) `ArrayList<String> a = new LinkedList<String>();`
False - There is no superclass-subclass relationship between `ArrayList` and `LinkedList`.
 - (b) `List<Integer> l = new List<Integer>();`
False - `List` is an interface
 - (c) `Set<Double> s = new TreeSet<Double>();`
True.
2. We generally assume that code inside of a **finally** block in a **try-catch** block will always be executed. Describe one situation in which this is not the case.
If the programmer calls `System.exit()` in the **try-catch** block. Alternatively, if an error serious enough to crash the program occurs before the **finally** block.
3. What is the difference between a class and an object?
A class is a definition for a type (members and methods). An object is an instance of a class.
4. What does the **super** keyword do? When must you use it?
The **super()** keyword is used to call the constructor of a super class. Use it at the beginning of the subclass constructor. It can also be used to call super class methods in cases where you have overridden a method and need the super class version. (`super.someMethod();`)
5. When do you have access to another object's private variables and methods? What two methods are likely to take advantage of this?
You only have access to another object's private variables within that object's class. Two methods most likely to take advantage of this are **equals** and **compareTo**.

6. Identify the errors in the following code:

Fixed.

```
1 public interface TestInterface {
2     public void splat( String message );
3 }
4 public abstract class AbstractClass {
5     abstract protected void myMethod();
6     public void callMyMethod() {
7         myMethod();
8     }
9 }
10 public class ConcreteClass extends AbstractClass implements TestInterface {
11     protected void myMethod() {
12         System.out.println( "I got called!" );
13     }
14     public void splat( String message ) {
15         System.out.println( "SPLAT!" );
16         System.out.println( message );
17     }
18 }
19 public class SimpleProgram {
20     public static void main(String[] args) {
21         AbstractClass myclass = new ConcreteClass();
22         myclass.callMyMethod();
23         //myclass.splat( "woohoo" ); // Not defined in AbstractClass
24     }
25 }
```

7. What are the two type of exceptions? What is an Error?

- (a) Checked - Code calling a method that may throw a checked exception must either provided a try-catch-(finally) block around the call OR declare that it throws the same exception in its method signature.
- (b) Runtime - Unanticipated software error that occurs at runtime and from which a program cannot usually recover.
- (c) Error - Serious issue outside of the program. Possibly a memory or hardware issue.

8. What will be printed by the following code?

```
1 public class ExceptionTest {
2     public static void main( String[] args ) {
3         try {
4             for ( int i = 0; i < 5; i++ ) {
5                 process( i );
6             }
7         } catch ( Exception e ) {
8             System.err.println( e.getMessage() );
9         } finally {
10            System.out.println( "Done processing" );
11        }
12    }
13    public static void process( int num ) throws Exception {
14        if ( num != 2 ) {
15            System.out.println( "Good number: " + num );
16        } else {
17            throw new Exception( "Bad number: " + num );
18        }
19    }
20 }
```

Good number: 0
Good number: 1
Bad number: 2
Done processing

9. Instantiate a new `BufferedReader` which decorates a `ForumReader`. `ForumReader`s only use the default constructor but can throw a `TooLongDidntReadException` if the forum post is too long to be worth reading.

```
1 try {
2     BufferedReader troll = new BufferedReader(new ForumReader());
3     // Do some things here.
4 } catch (TooLongDidntReadException tex) {
5     System.err.println(tex.getMessage());
6     // Cool story, bro!
7 } catch (Exception ex) {
8     System.err.println(ex.getMessage());
9     // Handle other exceptions.
10 } finally {
11     // Close the reader
12     if (troll != null) {
13         try {
14             troll.close();
15         } catch (IOException ex) {
16             ex.printStackTrace();
17         }
18     }
19 }
```

10. Why is buffering a good idea when doing file I/O?

Buffering uses an internal data structure to store data before reading/writing. Without

buffering, every read/write call would translate to a call at the OS level IO mechanism. Reading/writing a byte or so at a time is incredibly inefficient, so buffering stores data and then reads/writes the entire buffer. Buffering is drastically more efficient than not buffering.

11. Name and describe 3 Swing layout managers.

- (a) `GridLayout` - matrix representation (rows x columns)
- (b) `FlowLayout` - Default. Uses one row until it runs out of space.
- (c) `BorderLayout` - 5 sections: North, East, South, West and Center.

12. Describe what happens when you click on a `JButton`.

The button is notified of the click event and the button in turn notifies all of its listeners. Any attached `ActionListener` will have its `actionPerformed` method invoked.

13. Write an `ActionListener` implementation called `SimpleAction` that simply prints "action!" when an `ActionEvent` is received.

```
1 public class SimpleAction implements ActionListener {
2     public void actionPerformed(ActionEvent e) {
3         System.out.println("action");
4     }
5 }
```

14. Describe `List`, `Set` and `Map`. What interface do they all inherit?

`List` and `Set` implement `Collection`. `Map` does not implement any interface.

- (a) `List` - interface for an ordered collection that provides random access to elements
- (b) `Set` - interface for a collection containing no duplicate elements
- (c) `Map` - interface for a collection of key/value pairs.

15. Explain the difference between `Comparator` and `Comparable`.

- (a) `Comparable` - an interface implemented by a class to make it comparable to other objects, i.e. it specifies the natural ordering of instances of a particular class.
- (b) `Comparator` - an interface implemented by a class which can compare two objects of a particular type. This is useful for sorting algorithms in which the desired ordering is different than the natural ordering of the elements in the collection.

16. Use an iterator to count the number of elements in the `Collection`.

See `VictimsIterator.java`

17. Write a `Comparator` that compares strings by their size modulo (%) 13 and use this comparator for sorting a `TreeSet`.

See `ModuloComparator.java`

18. For each design pattern, briefly describe and how it works and why you would use it.
- (a) Model-view-controller
 - i. Description
 - Model: data/library classes
 - View: user interface classes
 - Controller: event handlers, facilitator between model and view
 - ii. Use - applications requiring intensive user interaction
 - (b) Observer
 - i. Description - An observer has a list of observables and notifies them of changes.
 - ii. Use - Event-driven programming and in MVC with action listeners
 - (c) Decorator
 - i. Description - Adding new functionality to existing class by wrapping objects under the hood.
 - ii. Use - Java IO classes. Reader/Writer and InputStream/OutputStream
 - (d) Iterator
 - i. Description - Gives access to members of a collection sequentially without revealing the underlying representation.
 - ii. Use - Collections classes that do not provide random/iterative access otherwise.
 - (e) Command
 - i. Description - An object encapsulates all of the information to execute at a later time.
 - ii. Use - Thread in Java simply has a **start** method, which encapsulates what will actually run.
 - (f) Factory
 - i. Description - Encapsulates how an object is created.
 - ii. Use - Factory is used when you have several implementations of a class/interface and you wish to not expose which instance gets created.
 - (g) Template Method
 - i. Description - Sets up the skeleton for an algorithm as an interface / abstract class and leaves the implementation up to deriving classes.
 - ii. Use - When you have a clear use case for inheritance and there will be some shared methods between the abstract class and each subclass.
 - (h) Composite
 - i. Description - Defining a type that contains a collection of instances of that type. Typically the type will have a derived “Leaf” type to signify the end of the composition. All instances are treated the same way as a single instance of the original type.
 - ii. Use - Structures that have a NonEmptyNode and an EmptyNode. A binary tree implementation could use a TreeNode interface from which Leaf inherits, but in the BinaryTree class we would only maintain a TreeNode root. TreeNode could also maintain left and right child TreeNode instances.